

1. Which of the following statements is correct for Java multi-threading programming:

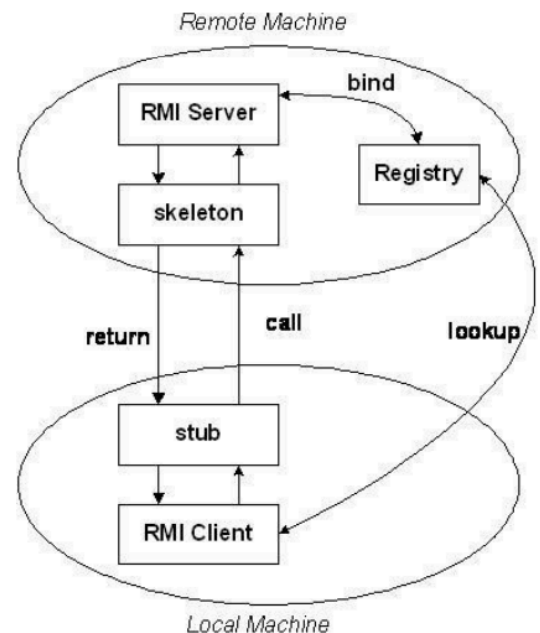
- A. A thread is in „runnable thread” state, if and only if, the programmer calls the start() method from an object which is instance from a class that is derived from the Thread class
- B. For a thread to be in "runnable thread" state, it is sufficient to create an object from a class that is derived from the Thread class
- C. For a thread to be in "runnable thread" state, it is sufficient to create an object from a class that is implementing the Runnable interface
- D. A thread is in "runnable thread" state, if and only if, the programmer calls the run() method from an object which is instance from a class that is derived from the Thread class
- E. A thread is in "runnable thread" state, if and only if, the programmer calls the stop() method from an object which is instance from a class that is derived from the Thread class

2. Please indicate which statement is correct:

- A. In the serialization process for an object is saved: the non-static field values, the value 0 or null for the static fields, field values which are not transient, the prototype and the body of the methods
- B. In the serialization process for an object is saved: the static field values, the field values which are not transient and the prototype of the methods
- C. In the serialization process for an object is saved: the static field values, the value 0 or null for the static fields, the field values which are transient, and the body and the prototype of the methods
- D. **In the serialization process for an object is saved: the non-static field values, the value 0 or null for the static fields, the field values which are not transient and the prototype of the methods**
- E. In the serialization process for an object is saved: the non-static field values, the field values which are not transient and the body of the methods ***

3. What is the true statement for Java RMI 1.1 in terms of the logical flow actions' order:

- A. **Sequence: 1. bind, 2. lookup, 3. call, 4. return**
- B. Sequence: 1. call, 2. lookup, 3. bind, 4. return
- C. Sequence: 1. lookup, 2. call, 3. bind, 4. return
- D. Sequence: 1. lookup, 2. bind, 3. call, 4. return
- E. Sequence: 1. call, 2. return, 3. bind, 4. lookup



4. Knowing that at the offset 0x44 in the following SNMP traffic analysis is starting the OID – Object Identifier – value and knowing that the ASN.1 DER tag for OIDs declaration is 0x06:-Please specify which OID value is correct for the byte array sequence – the selected sequence: 0x30 0x0B 0x06 0x07 0x2B 0x06 0x01 0x02 0x01 0x01 0x03 0x05 0x00:

No.	Time	Source	Destination	Protocol	Info
1	0.000000	10.2.5.146	10.2.4.198	SNMP	get-next-request SNMPv2-MIB::sysUpTime
2	0.000682	10.2.4.198	10.2.5.146	SNMP	get-response SNMPv2-MIB::sysUpTime.0

+ Frame 1 (81 bytes on wire, 81 bytes captured)
 + Ethernet II, Src: Usi_43:1e:dc (00:16:41:43:1e:dc), Dst: Fujitsu_70:75:14 (00:17:42:70:75:14)
 + Internet Protocol, Src_ 10.2.5.146 (10.2.5.146), Dst: 10.2.4.198 (10.2.4.198)
 + User Datagram Protocol, Src Port: nbx-au (2094), Dst Port: snmp (161)
 — Simple Network Management Protocol

version: version-1 (0)
 community: public
 — data: get-next-request (1)
 — get-next-request
 request-id: 1
 error-status: noError (0)
 error-index: 0

+ variable-bindings: 1 item

```

0000  00 17 42 70 75 14 00 16 41 43 1e dc 08 00 45 00
0010  00 43 92 59 00 00 80 11 89 f5 0a 02 05 92 0a 02
0020  04 c6 08 2e 00 a1 00 2f a8 2a 30 25 02 01 00 04
0030  06 70 75 62 6c 69 63 a1 18 02 01 01 02 01 00 02
0040  01 00 30 0d 30 0b 06 07 2b 06 01 02 01 01 03 05
0050  00
  
```

- A. 1.3.6.1.2.1.1.3.5
- B. 1.3.6.1.2.1.1.3.5.0
- C. 1.2.1.1.3.5
- D. **1.3.6.1.2.1.1.3**
- E. None

5. The result of this program is:-After the run phase which statement is true:

```

package eu.ase.reflection;
import java.lang.reflect.*;
public class DumpMethods5{
    public static void main(String[] args){
        try{
            Class c = Class.forName(args[0]);
            Method[] m = c.getDeclaredMethods();
            for(int i = 0; i < m.length; i++)
                System.out.println(m[i].toString());
        }
        catch(Throwable e){
            System.err.println(e);
        }
    }
}
  
```

- A. Display all the fields of the class that its name is passed as argument for the program in args[0]
- B. Display static and nonstatic only public methods' prototypes of the class that its name is passed as argument for the program in args[0]
- C. Display static and nonstatic only private methods' prototypes of the class that its name is passed as argument for the program in args[0]
- D. Display all the static and nonstatic methods' prototypes of the class that its name is passed as argument for the program in args[0]**

6. Taking into account the following JSP source code:-Please specify which statement is true if there's the class "eu.ase.beans.FormularBean" bytecode:

```
<HTML> <HEAD><TITLE> JSP Response for processing the card </TITLE></HEAD>
<BODY>
<jsp:useBean id = "fb1" class = "eu.ase.beans.FormularBean" Scope = "session"
...</BODY></HTML>
```

- A. for each browser that accesses the JSP page, it is created an "fb1" object from the class "eu.ase.beans.FormularBean", for first HTTP request**
- B. for all the browser clients accessing the JSP page, it is created only a single object "fb1" from the class "eu.ase.beans.FormularBean", for first HTTP request
- C. for all the browser clients accessing the JSP page, it is created only a single object "fb1" from the class "eu.ase.beans.FormularBean", for first HTTP request
- D. for each browser that accesses the JSP page, it is created an "fb1" object from the class "eu.ase.beans.FormularBean", for every HTTP request
- E. None

7. Taking into account the stateless HTTP protocol analysis between the web browser– Mozilla and the Java Servlet from the server side:

```
GET /S09/ShowSession HTTP/1.1
Accept: */*
Accept-Language: ro
UA-CPU: x86
Accept-Encoding: gzip, deflate
User-Agent: Mozilla/4.0
Host: 192.168.100.19:8080
Connection: Keep-Alive
Cookie: CookieIon=IONIONION
```

```
HTTP/1.1 200 OK
Server: Apache-Coyote/1.1
Set-Cookie: JSESSIONID=CBC58386A4D14F02F2C3F4F0895BB9E3; Path=/S09
Content-Type: text/html
Content-Length: 547
Date: Tue, 07 Apr 2009 11:42:32 GMT
```

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.0 Transitional//EN">
<HTML>
<HEAD><TITLE> Show Session</TITLE></HEAD>
```

And knowing the source code for the Java Servlet which is processing the HTTP GET method:

```

public void processRequest(HttpServletRequest request, HttpServletResponse response){
    HttpSession session = request.getSession(true);
    response.setContentType("text/html");
    PrintWriter out = response.getWriter();
    String title = "Show Session";
    String heading;
    Integer accessCount = new Integer(0);
    if(session.isNew()){
        Heading = "Welcome, Newcomer";
        session.setAttribute("accessCount", accessCount);
    }
    else{

```

Please specify which statement is true:

- A. The Java Servlet with ShowSession name from the S09 context of the server with IP: 192.168.100.19 and port: 8080, processed the "if (session.isNew ())" branch in method "processRequest" and set the "cookie" JSESSIONID=CBC5... for storing variable "accessCount" on the current session. The variable "accessCount" is visible for each client that makes HTTP GET request to the servlet
- B. The Java Servlet with ShowSession name from the S09 context of the server with IP: 192.168.100.19 and port: 8080, processed the "if (session.isNew ())" branch in method "processRequest" and set the "cookie" JSESSIONID=CBC5... for storing variable "accessCount" on the current session. The variable "accessCount" is visible (is the "same") for all the clients that make HTTP GET request to the servlet
- C. The Java Servlet with ShowSession name from the S09 context of the server with IP: 192.168.100.19 and port: 8080, processed the "else" branch in method processRequest
- D. The Java Servlet with ShowSession name from the S09 context of the server with IP: 192.168.100.19 and port: 8080, processed the "if (session.isNew ())" branch in method "processRequest" and set the "cookie" CookieName = IONIONION for storing variable "accessCount" on the current session. The variable "accessCount" is visible (is the "same") for all the clients that make HTTP GET request to the servlet
- E. The Java Servlet with ShowSession name from the S09 context of the server with IP: 192.168.100.19 and port: 8080, processed the "if (session.isNew())" branch in method "processRequest" and set the "cookie" CookieName = IONIONION for storing variable "accessCount" on the current session. The variable "accessCount" is visible for each client that makes HTTP GET request to the servlet

8. For the Base 64 we have the following encoding table:-The „FTP” word has in ISO-8859-1 the following hex value: 0x46 0x54 0x50 – please specify what value is true for Base64:

Base 64 Code

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
00	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25

a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51

0	1	2	3	4	5	6	7	8	9	+	/	=
52	53	54	55	56	57	58	59	60	61	62	63	64 (64 is used as padding)

- A. RI RQ = 0x11 0x25 0x11 0x10
- B. SLSQ = 0x12 0x0B 0x12 0x10

- C. None of the presented answers
- D. RLRQ = 0x11 0x0B 0x11 0x10
- E. SkSQ = 0x12 0x24 0x12 0x10

9. Speaking about the development of a TOP-DOWN web service:

- A. the starting point is an already existing WSDL file**
- B. the WSDL file is created during web service development
- C. there is no such web service type
- D. there is no WSDL file involved

10. Speaking about the development of a BOTTOM-UP web service:

- A. there is no WSDL file involved
- B. there is no such web service type
- C. the WSDL file is created during web service development**
- D. the starting point is an already existing WSDL file

11. After the traffic analysis for FTP on port 21, we have the following network packet:-Please specify the true statement:

- ✖ Frame 20 (79 bytes on wire, 79 bytes captured)
- ✖ Ethernet II, Src: IntelCor_cd:71:7f (00:13:e8:cd:71:7f), Dst: Giga-Byt_4a:6c:47 (00:14:85:4a:6c:47)
- ✖ Internet Protocol, Src: 192.168.0.5 (192.168.0.5), Dst: 192.168.0.1 (192.168.0.1)
- ✖ Transmission Control Protocol, Src Port: 1476 (1476), Dst Port: ftp (21), Seq: 39, Ack: 136, Len: 25
- ✔ File Transfer Protocol (FTP)

 ✔ PORT 192,168,0,5,19,137\r\n

 Request command: PORT

 Request arg: 192,168,0,5,19,137

```

0000  00 14 85 4a 6c 47 00 13 e8 cd 71 7f 08 00 45 00
0010  00 41 1b 85 40 00 80 06 5d db c0 a8 00 05 c0 a8
0020  00 01 05 c4 00 15 39 6e f4 05 7e 8c bd 19 50 18
0030  b3 84 7d 71 00 00 50 4f 52 54 20 31 39 32 2c 31
0040  36 38 2c 30 2c 35 2c 31 39 2c 31 33 37 0d 0a

```

- A. Carry out an "active" FTP data transmission from the client (192.168.0.5) to the FTP server (192.168.0.1) by negotiating with the client TCP port 137
 - B. Carry out an "active" FTP data transmission from the client (192.168.0.5) to the FTP server (192.168.0.1) by negotiating the client TCP port 19137
 - C. Carry out an "active" FTP data transmission from the client (192.168.0.5) to the FTP server (192.168.0.1) by negotiating with the client TCP port 19
 - D. Perform a "passive" FTP data transmission from the server (192.168.0.5) to the FTP client (192.168.0.1) by negotiation the server TCP port 19137
 - E. Carry out an "active" FTP data transmission from the client (192.168.0.5) to the FTP server (192.168.0.1) by negotiating with the client TCP port 5001**
-

12. În urma rulării programului următor:

```
package eu.ase;
public class Certificate {
    private int uid;
    private String issuerName;
    public Certificate() {
        this.uid = 0;
        this.issuerName = null;
    } public

    Certificate(int uidp, String issuerNamep) {
        this.uid = uidp;
        this.issuerName = issuerNamep;
    }

    public void setUid(int c) {
        this.uid = c;
    }

    public void setIssuerName(String n) {
        this.issuerName = n;
    }

    public String toString() {
        return new String("\tUid = "+this.uid+" , Issuer =" +this.issuerName);
    }
} //end class Certificate
public class ProgMain {
    public static void main(String args[]) {
        Certificate a = new Certificate(234, "Ion");
        Certificate b = new Certificate(355, "Vasile");
        b = a; b.setUid(888);
        b.setIssuerName("WWW");
        System.out.print("\n a="+a.toString());
        System.out.print("\n b="+b.toString());
    }
}
```

ce mesaje sunt afișate pe ecran?

- A. a= Uid = 888, Issuer = WWW si b= Uid = 888, Issuer = WWW
- B. a= Uid = 234, Issuer = WWW si b= Uid = 888, Issuer = WWW
- C. a= Uid = 234, Issuer = Ion si b= Uid = 355, Issuer = WWW
- D. a= Uid = 234, Issuer = Ion si b= Uid = 888, Issuer = WWW

13. Se consideră clasele:

```
class Baza{
    public int[] valori1;
    public Baza(){
        System.out.println("Apel CBI");
        valori1 = new int[5];
    }
    public Baza(int n){
        valori1 = new int[n];
        System.out.println("Apel CB");
    }
}
class Derivat extends Baza{
    public int[] valori2;
    public Derivat(int n){
        valori2 = new int[n];
        System.out.println("Apel CD");
    }
}
```

Execuția programului următor are ca efect:

```
public class subiect2{
    public static void main(String args[]){
        Baza b = new Baza(5);
        Derivat d = new Derivat(6);
    }
}
```

- A. Se obțin mesajele Apel CBI; Apel CB; Apel CD;
- B. Se obțin mesajele Apel CB; Apel CBI; Apel CD;**
- C. Se obțin mesajele Apel CD; Apel CB;
- D. Se obțin mesajele Apel CB; Apel CD;

14. Conceptul de polimorfism se implementează prin:

- A. Supraincarcare de functii si prin virtualizare
- B. Supraincarcare si derivare/mostenire de clase
- C. Supradefinire si virtualizare de functii
- D. Supraincarcare si supradefinire de functii**

15. Se considera programul:

Raf.seek(argument) -> argument este **byte**-ul de la care incepe citirea. Va afisa **56**.

Argument = **0** -> afiseaza **10**

Argument = **4** -> afiseaza **15**

Argument = **8** -> afiseaza **2**

Argument = **12** -> afiseaza **56**

```
public class subiect4{
    public static void main(String args[]){
        int sir_valori[] = {10,15,2,56,67,5};
        try{
            RandomAccessFile raf = new RandomAccessFile("test.dat", "rw");
            for (int i = 0; i < sir_valori.length; i++)
                raf.writeInt(sir_valori[i]);
            raf.seek(12);
            System.out.println("Valoare gasita:" + raf.readInt());
            raf.close();
        }
        catch (IOException e){
            System.out.println("Eroare acces fisier");
        }
    }
}
```

Prin compilarea si executia programului se obtine:

- A. Se afiseaza valoarea 2;
- B. Se afiseaza valoarea 56;**
- C. Se afiseaza valoarea 67;
- D. Se afiseaza valoarea 15;
- E. Niciuna din valorile de mai sus

16. What output maybe generated by the following C/C++ OpenMP source code - be aware that are 3 threads and the iterations from 0 to 4?

```
#include <stdlib.h>;
#include <stdio.h>;
#include "omp.h";
int main()
{
    int nthreads, tid;
    omp_set_num_threads(3);
    int i;
    #pragma omp parallel private(tid) shared(i)
```



```

{
    tid = omp_get_thread_num();
    printf("Hello world from (%d)\n", tid);
    #pragma omp for
    for(i = 0; i <= 4; i++)
    {
        printf(" Iteration %d by (%d)", i, tid);
    }
} //all threads join master thread and terminate
return 0;
}

```

- A. Hello world from (0) Hello world from (2) Iteration 4 by (2) Iteration 0 by (0) Iteration 1 by (0) Hello world from (1) Iteration 2 by (1) Iteration 3 by (1) Iteration 5 by (1)
- B. Hello world from (0) Hello world from (2) Iteration 4 by (2) Iteration 0 by (0) Iteration 1 by (0) Hello world from (1) Iteration 2 by (1) Iteration 3 by (1)**
- C. Hello world from (0) Hello world from (2) Iteration 4 by (2) Iteration 0 by (0) Iteration 1 by (0) Hello world from (1) Iteration 2 by (1) Iteration 3 by (1) Iteration 5 by (2)
- D. Hello world from (0) Iteration 5 by (0) Hello world from (2) Iteration 4 by (2) Iteration 0 by (0) Iteration 1 by (0)

17. What statement is true for the following C/C++ OpenMP source code?

```

#include <iostream>
#include <vector>
#include <omp.h>
using namespace std;
#include <time.h>
clock_t start, stop;
int main(int argc, char** argv)
{
    int nr_elements = 1000000, i = 0, nrThreads = 4;
    vector<int> u1(nr_elements, 1),
    u2(nr_elements, 2);
    vector<int> result(nr_elements, 0);
    vector<int> resulttmp(nr_elements, 0);
    //nrThreads = atoi(argv[1]);
    start = clock();
    for (i = 0; i < nr_elements; i++)
    {
        result[i] = u1[i] + u2[i];
    }
    stop = clock();
    for (int i = 0; i < 20; i++)

```

```

        cout << " , " << result[i];
        cout << "It took " <<(stop - start) << " clicks (" <<
(((float)(stop - start)) / CLOCKS_PER_SEC) << "seconds)." << endl;

// a horkshoring for construct to add vectors:
omp_set_num_threads(nrThreads);
start = clock();
#pragma omp parallel for
for (i = 0; i < nr_elements; i++)
{
    resulttmp[i] = u1[i] + u2[i];
}
stop = clock();
cout << "It took " <<(stop - start) << " clicks (" << (((float)(stop
- start)) / CLOCKS_PER_SEC) << "seconds)." << endl;
for (int j = 0; j < 20; j++)
    cout << " , " << resulttmp[j];
return 0;
} //end main

```

- A. The program is adding 2 arrays/Vectors using 2 threads
- B. The program is adding 4 arrays/vectors using 4 threads
- C. The program is not using OpenMP syntax
- D. The program is adding 4 arrays/vectors using 4 threads
- E. None of the existing answers

18. What statement is true taking into account the following Java EE source code in terms ofJMS -Java Messaging Service API?

```

package eu.ase.jms;
import javax.jms.*;
import javax.naming.*;
public class QueueReceiver{
public static void main(String[] args){
    String queueName = null;
    Context jndiContext= null;
    QueueConnectionFactory queueConnectionFactory= null;
    QueueConnection queueConnection = null;
    QueueSession queueSession = null;
    Queue queue = null;
    QueueReceiver queueReceiver= null;
    TextMessage message = null;
    queueName = new String(queue/testQueue);
    try{

```

```

        jndiContext= new InitialContext();
    } catch (NamingException e){
        System.out.println("Could not create JNDI API" + "context:" +
e.toString());
    }
    try {
        queueConnectionFactory=
        (QueueConnectionFactory)InitialContext.lookup('UILConnectionFactory");
        queue = (Queue) jndiContext.lookup(queueName);
    } catch (NamingException e){
        System.out.println('JNDI API lookup failed:" + e.toString():
        System.exit(1);
    }
111
    ,{
        queueConnection = queueConnectionFactory.createQueueConnection();
        queueSession = queueConnection.createQueueSession(false,
Session.AUTO_ACKNOWLEDGE);
        queueReceiver= queueSession.createReceiver(queue);
        queueConnection.start()
        while (true){
            Message m = queueReceiver.receive(1);
            if (m != null){
                if (m instanceof TextMessage){
                    message = (TextMessage) m;
                    System.out.println('Reading message:" +
message.getText());
                } catch (NamingException e){
                    System.out.println("JNDI API lookup failed:" + e.toString());
                    System.exit(1);
                }
            }
            try{
                queueConnection = queueConnectionFactory.createQueueConnection();
                queueSession = queueConnection.createQueueSession(false,
Session.AUTO_ACKNOWLEDGE);
                queueReceiver= queueSession.createReceiver(queue);
                queueConnection.start()
                while (true){
                    Message m = queueReceiver.receive(1);
                    if (m != null){
                        if (m instanceof TextMessage){
                            message = (TextMessage) m;
                            System.out.println('Reading message:" + message.getText());
                        } else {
                            break;
                        }
                    }
                }
            } catch (NamingException e){
                System.out.println("JNDI API lookup failed:" + e.toString());
                System.exit(1);
            }
        }
    }
}

```

```

}
} catch (JMSException e){
    System.out.println('Exception occurred:' + e.toString());
}finally(
    if (queueConnection != null){
        try{
            queueConnedion.close();
        } catch (JMSException e){}
    }end main
} //end class

```

- A. The client program consumes/receives messages from a Java EE server queue and it is looping forever.
- B. The client program consumes/receives messages from a Java EE server queue and it is breaking the while loop if and only if it receives a text message
- C. The source code is not compliant with JMS API
- D. D. None of the existing answers
- E. The client program consumes/receives messages from a Java EE server queue and it is breaking the while loop if and only if it receives a non-text message.**

19. What statement is true with respect to the following Java EE - RESTful Service class source code (using Jersey API):

```

package eu.ase.rest;
import javax.ws.rs.GET;
import javax.ws.rs.Path;
import javax.ws.rs.Produces;
import javax.ws.rs.core.MediaType;
@Path("UserService") public class UserService {
    UserDao userDao = new UserDao();
    @GET @Path("/users")
                                @Produces(MediaType.APPLICATION_XML)
    //@Produces(MediaType.APPLICATION_JSON) public List<User> getUsers()
        return userDao.getAllUsers();
    }
} //Bend class

```

- A. It is a RESTful class which handles HTTP GET requests for getUsers resource**
- B. None of the existing answers
- C. It is not a correct Java source code
- D. It is a RESTful class which handles both HTTP GET and POST requests for getUsers resource
- E. It is a RESTful class which handles both HTTP POST requests for getUsers resource

20. SUN Java CORBA has 3 parts:

- A. None of the existing answers
- B. — ORB (Object Request Broker) in Class/JAR
 - Naming Service: rmiregistry for both Non-persistent & Persistent
 - IDL idltojava & javatoidl compiler— now: idlj.exe
- C. — ORB (Object Request Broker) in Class/JAR**
 - Naming Service: COS — CORBA Object Service: tnameserv.exe (Non-persistent) & orbd.exe (Persistent)**
 - IDL idltojava & javatoidl compiler— now: idlj.exe**
- D. — ORB (Object Request Broker) as OS Service/Process
 - Naming Service: COS
 - CORBA Object Service: tnameserv.exe (Non-persistent) & orbd.exe (Persistent)
 - IDL idltojava & javatoidl compiler— now: idlj.exe
- E. — ORB (Object Request Broker) as OS Service/Process
 - Naming Service: COS
 - CORBA Object Service: only orbd.exe
 - IDL idltojava & javatoidl compiler— now: idlj.exe

21. What statement is not true for OpenMP programming model?

- A. OpenMP programming model uses Explicit Parallelism: Programmer has full control over parallelization. OpenMP is not an automatic parallel programming model.
- B. OpenMP programming model uses shared memory, thread-based parallelism: OpenMP is based on the existence of multiple threads in the shared memory programming paradigm. (A shared memo)
- C. OpenMP programming model is designed to guarantee that input or output to the same file is synchronous when executed in parallel.**
- D. OpenMP programming model uses Compiler directive based: Most OpenMP parallelism is specified through the use of compiler directives which are embedded in the source code.
- E. None of the existing answers

22. What statement is true for OpenMP?

- A. None of the existing answers.**
- B. OpenMP is implemented identically by all vendors
- C. OpenMP is meant for distributed-memory parallel systems
- D. OpenMP is guaranteed to make the most efficient use of shared memory
- E. OpenMP is meant to cover compiler-generated automatic parallelization and directives to the compiler to assist such parallelization

23. Which statement is NOT true for CORBA - Common Object Request Broker Architecture - in terms of the concepts?

- A. In CORBA is mandatory to have both the clients and servants in Java.
- B. None of the existing answers
- C. CORBA Skeleton lives on server
- D. CORBA Skeleton receives requests from stub via ORB
- E. CORBA Stub lives on client

24. If the number of OpenMP threads is set to 4 (OMP_NUM_THREAD = 4). What would be a potential, output of the following Open MP Parallel source code:

```
include <stdlib.h>
include <stdio.h>
include <omp.h>
int main ()
{
    pragma omp parallel
    {
        int ID = omp_get_thread_num();
        printf("Hello (%d)\n", ID);
        printf("World (%d)\n", ID);
    }
    return 0;
}
```

- A. Hello (0) World (0) Hello (2) World (2) Hello (1) World (1) Hello (2) World (2)
- B. Hello (0) World (0) Hello (0) World (1) Hello (1) World (2) Hello (2) World (3)
- C. Hello (0) world (1) Hello (2) world (3)
- D. None of the existing answers
- E. Hello (0) World (0) Hello (2) World (2) Hello (1) World (1) Hello (3) World (3)

1. Q: embarrassingly parallel workload problem

- A. little or no effort is needed to split the problem into a number of parallel tasks (nu asta era raspunsul, dar trebuie stiut conceptual)

2. When is MPI_Recv() method called?

3. Know what OpenMP is and is not

4. JMS is asynchronous and RMI is synchronous (variantele erau lungi, dar asta e idee)

5. Considering the next sequence is executed on 4 threads in parallel, indicate what value it will display:

```
#include <iostream>
#include "omp.h"
void main() {
    int s = 0;
    #pragma omp parallel for reduction(+:s)
        for (int i = 0; i < 4; i++) {
            s++;
        }
    std::cout << std::endl << " s = " << s;
}
```

- A. 24
- B. 16
- C. Unable to determine because variable "s" is shared and there will be a Race condition issue
- D. 0
- E. 4

6. Amdahl's Law - o grila cu ce e adevarat si o grila cu ce e fals despre concept

7. SISD, MISD, SIMD, MIMD

8. JMS how many messaging patterns? Two, point-to-point & publish/subscribe

9. Q: Using nowait in an OpenMP for, what is the effect?

- A. Removes the block of the for structure

10. What does nowait do in OpenMP?

11. Multe grile cu OpenMPI si ce afiseaza programul dupa rulare

12. Calcul OID din ASN.1 (vezi regula de calcul)

13. Q: Message types in Java EE?

- A. TextMessage, MapMessage, BytesMessage, StreamMessage, ObjectMessage, Message